OR Spectrum (2004) 26: 75–92
DOI: 10.1007/s00291-003-0140-8

OR Spectrum

© Springer-Verlag 2004

The berth allocation problem: models and solution methods

Yongpei Guan[1] **and Raymond K. Cheung**[2]

[1] School of Industrial and Systems Engineering, George Institute of Technology, Atlanta, GA 30332, USA (e-mail: guanyp@isye.gatech.edu)
[2] Department of Industrial Engineering and Engineering Management, The Hong Kong University of Science and Technology, Clear Water Bay, Kowloon, Hong Kong (e-mail: rcheung@ust.hk)

Abstract. In this paper, we consider the problem of allocating space at berth for vessels with the objective of minimizing total weighted flow time. Two mathematical formulations are considered where one is used to develop a tree search procedure while the other is used to develop a lower bound that can speed up the tree search procedure. Furthermore, a composite heuristic combining the tree search procedure and pair-wise exchange heuristic is proposed for large size problems. Finally, computational experiments are reported to evaluate the efficiency of the methods.

Keywords: Berth allocation – Tree search procedure – Heuristics

We consider the problem of allocating berth space for vessels in container terminals, which we refer to as the *berth allocation problem*. The research is motivated by such a problem in Hong Kong. Being the world's busiest container ports in most of the last decade, Hong Kong had a container throughput of over 18 million twenty-foot equivalent units (TEU) in 2002. As the berth space is very limited in Hong Kong and thousands of containers have to be handled everyday, an effective berth allocation is critical to the efficient management of container traffic flow. A typical berth at the container terminals can accommodate multiple vessels at the same time. When no berth space is available, a vessel needs to wait for mooring. For simplicity, we call the sum of the waiting time and the processing time of a vessel as its *flow* time. Our objective is to allocate berth space to vessels and to schedule the vessels such that the total weighted flow time is minimized, where the weights reflect the relative importance of vessels.

* The authors would like to thank the helpful comments of two anonymous referees and the editors. The research was supported in part by Grant HKUST6039/01E of the Research Grant Council of Hong Kong

Correspondence to: R.K. Cheung

Several berth allocation models have appeared in the literature. They differ in the assumptions being made, such as whether vessel waiting is allowed, whether multiple vessels mooring on a berth is possible, whether vessel arrival times are considered, and whether the processing times are proportional to vessel size. Models that assume no vessel waiting are considered in Chen and Hsieh [1], Brown et al. [2] and Lim [12]. Vessels do not need to wait when berth space is abundant or parallel mooring is possible. Chen and Hsieh [1] develop an integer programming model in the form of generalized network with side constraints to minimize the total berthing cost. The costs in the model reflect the distances between the vessel mooring locations and the container locations in a terminal. Brown et al. [2] study the berthing of submarines at a naval base where parallel mooring and shifting of submarines along the berth are allowed. The problem is formulated as an integer program that minimizes the total berthing cost. For commercial ports, however, parallel mooring and shifting of vessels along berths are rare. On the other hand, Lim [12] considers the problem of minimizing the total berth length used for a given set of vessels. The problem is modeled as a network and the decision variables are the directions of arcs, which indicate the relative positions of vessels along the berth.

Models that permit vessel waiting are considered in [3,4,7–11,13]. Imai et al. [7] assume that only one vessel is moored to a berth at a time and develop a model for scheduling the currently waiting vessels to moor so as to minimize the total flow time. Imai et al. [8] extend the model by considering both the currently waiting vessels and the incoming vessels. A Lagrangian relaxation based procedure is developed. Allowing multiple vessels to be serviced at a berth simultaneously, Daganzo [3] discusses several principle-based heuristics for scheduling shore cranes so as to minimize the vessel waiting time, whereas Peterkofsky and Daganzo [13] develop a branch-and-bound method to solve such a problem. Lai and Shih [10] use simulation models to evaluate different berthing policies. The numerical results indicate that different policies may be used for different vessel arrival patterns. Kim and Moon [9] develop a mixed-integer programming model for the berth scheduling problem. The objective is to minimize the costs resulting from berthing vessels at the non-ideal locations and the delay costs for the vessel departures. When the arrival times of vessels are not considered, Li et al. [11] study the berth allocation problem where the processing time of a vessel is agreeable with the vessel size. That is, a larger vessel requires a longer processing time. The paper proposes a multiple-job-on-one-processor scheduling model with the objective of minimizing the latest vessel departure time (or makespan in scheduling terminology). A heuristic is developed of which the worst case analysis is performed. Finally, Guan et al. [4] consider a similar model but with the objective of minimizing the total weighted flow time. The problem is formulated as a multi-processor-for-one-job scheduling model and a new heuristic is proposed and analyzed.

In this paper, we study a more general berth allocation model that allows multiple vessel mooring per berth, considers vessel arrival times, and has the objective of minimizing the total weighted flow time. First, we consider two mathematical formulations for the model. The first one is similar to the model of [9] but with a slightly different objective. Our formulation has a different objective and the for-

mulation is used to develop a tree search procedure for obtaining an exact solution of the problem. The other formulation is used to develop a tight lower bound that can speed up the tree search procedure. Next, we develop a composite heuristic in which the tree search procedure developed can be combined with the heuristic of Guan et al. [4] and a simple pair-wise exchange procedure. Finally, we conduct numerical experiments for evaluating the efficiency and the effectiveness of the methods.

The remainder of this paper is organized as follows: Section 1 presents the two mathematical formulations. Section 2 describes a tree search procedure while Section 3 introduces the heuristics. Section 4 reports the numerical results. Finally, Section 5 summarizes our research and outlines some potential research directions.

1 Formulations

In our model, we discretize the continuous time and space into integer units. The sections of the berth are indexed by $1, 2, \ldots, S$, the time units over the planning horizon are indexed by $1, 2, \ldots, T$, and the vessels are indexed by $1, 2, \ldots, N$. We use the notion of $[a, b]$ to represent the set of integers between and including a and b. For each vessel i, define

p_i = Processing time for vessel i,
s_i = Size of vessel i, measured in the number of berth sections and $s_i \leq S$,
a_i = Arrival time of vessel i,
w_i = Weight assigned for vessel i.

We make a number of assumptions in our model:

1. Vessel arrivals can be grouped into batches. Our model is for tactical planning where container shipping companies provide approximate vessels' arrival times in the format of time ranges, such as the afternoon of a particular day. We group the vessels with a similar arrival time range as a batch and set the arrival times of this group of vessels as the beginning of the time range.
2. Once a vessel is moored, it will remain in its location until all the required container processing is done. In practice, any interruption of the container processing during mooring is costly. Thus, this assumption is valid in practice.
3. Vessel processing times and vessel sizes are agreeable, meaning that if $s_i \geq s_j$, then $p_i \geq p_j$, and vice versa. This assumption reflects that a larger vessel requires a longer processing time.
4. A berth section handles at most one vessel at a time. Multiple mooring of vessels at a section happens at specialized terminals (such as submarine base or barge terminal) but is not common for large ocean-going terminals.

The berth allocation problem can be represented by a time-space diagram where the horizontal axis and the vertical axis represent the time units and berth sections respectively. A vessel can be viewed as a rectangle whose length is the processing time and whose height is the vessel size. We call vessel i mooring at berth section ℓ and time t, then the vessel occupies the consecutive berth sections between ℓ and $\ell + s_i - 1$ and from time units t to $t + p_i - 1$. Let

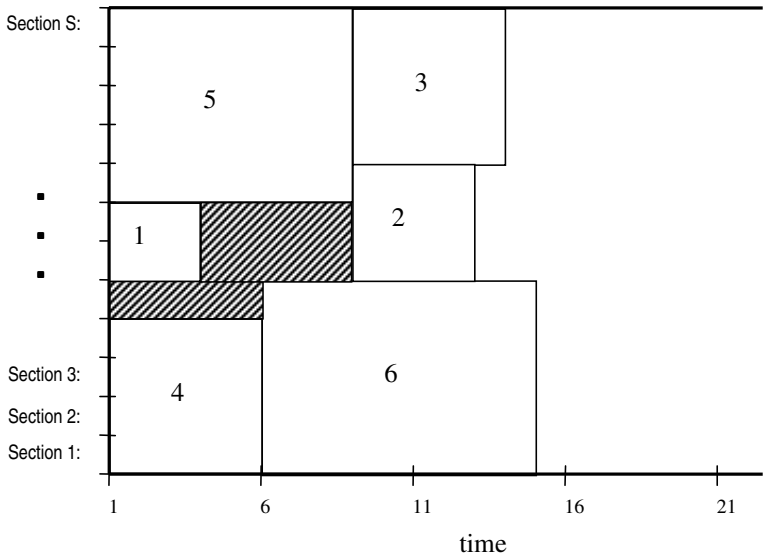


Fig. 1. Time-space representation

u_i = The mooring time of vessel i,
v_i = The starting berth section occupied by vessel i,
c_i = The departure time of vessel i.

We say that vessel rectangle i is immediately *left* of vessel rectangle j if $c_i = u_j$ (that is, vessel j is moored right after vessel i is finished) and vessel rectangle i is immediately *above* vessel rectangle j if $v_i = v_j + s_j$. Figure 1 shows the time-space diagram of 6 vessels.

1.1 Relative position formulation

One way to represent the problem is to consider the relative position of the vessel rectangles in the time-space diagram. Let

$$\sigma_{ij} = \begin{cases} 1, & \text{if vessel rectangle } i \text{ is completely on the left of vessel rectangle } j \\ & \text{and the two rectangles are not overlapped,} \\ 0, & \text{otherwise;} \end{cases}$$

$$\delta_{ij} = \begin{cases} 1, & \text{if vessel rectangle } i \text{ is completely below vessel rectangle } j \text{ and} \\ & \text{the two rectangles are not overlapped,} \\ 0, & \text{otherwise.} \end{cases}$$

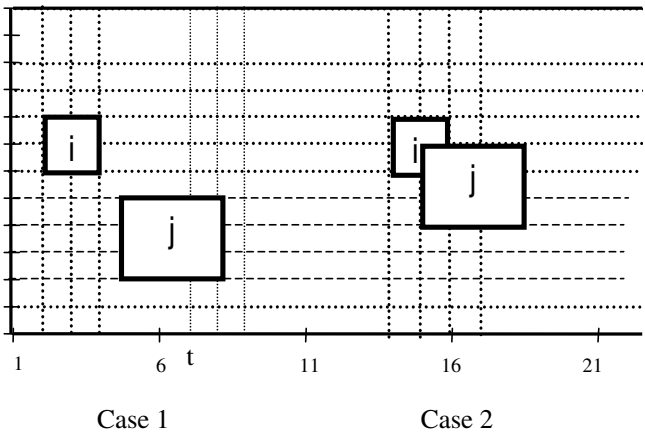


Fig. 2. Relative position formulation

The mathematical formulation of the berth allocation can be written as:

$$\min \sum_{i=1}^{N} w_i(c_i - a_i) \tag{1}$$

subject to

$$u_j - u_i - p_i - (\sigma_{ij} - 1)T \geq 0 \qquad \forall i, j \tag{2}$$

$$v_j - v_i - s_i - (\delta_{ij} - 1)S \geq 0 \qquad \forall i, j \tag{3}$$

$$\sigma_{ij} + \sigma_{ji} + \delta_{ij} + \delta_{ji} \geq 1 \qquad \forall i, j \tag{4}$$

$$\sigma_{ij} + \sigma_{ji} \leq 1 \qquad \forall i, j \tag{5}$$

$$\delta_{ij} + \delta_{ji} \leq 1 \qquad \forall i, j \tag{6}$$

$$p_i + u_i = c_i \qquad \forall i \tag{7}$$

$$u_i \in [a_i, T - p_i + 1], v_i \in [1, S - s_i + 1] \qquad \forall i \tag{8}$$

$$\sigma_{ij} \in \{0, 1\}, \delta_{ij} \in \{0, 1\} \qquad \forall i, j \tag{9}$$

We refer to the formulation defined by (2)–(8) as the Relative Position Formulation (RPF). Constraints (2)–(3) enforce the definitions of σ_{ij} and δ_{ij}. To see that, suppose that vessel rectangle i is on the left of vessel rectangle j. By definition, $\sigma_{ij} = 1$ and (2) becomes $u_j \geq u_i + p_i$. This reflects the requirement that vessel j cannot be started until vessel i is finished. Conversely, if vessel rectangle i is not on the left of vessel rectangle j, then the requirement is not needed. In this case, $\sigma_{ij} = 0$ and (2) becomes redundant since $u_j + T \geq u_i + p_i$ is always satisfied. Constraints (4)–(6) ensure that no vessel rectangles overlap. For example, in case 1 of Figure 2, rectangle i is at the left-and-above position of rectangle j and thus $\sigma_{ij} = 1$, $\sigma_{ji} = 0$, $\delta_{ij} = 0$ and $\delta_{ji} = 1$. In case 2 of Figure 2, the rectangles are overlapped and the values of all σ_{ij}, σ_{ji}, δ_{ij}, and δ_{ji} are all 0. In this situation, constraint (4) is violated. Constraint (7) defines the departure time as the vessel mooring time plus the processing time.

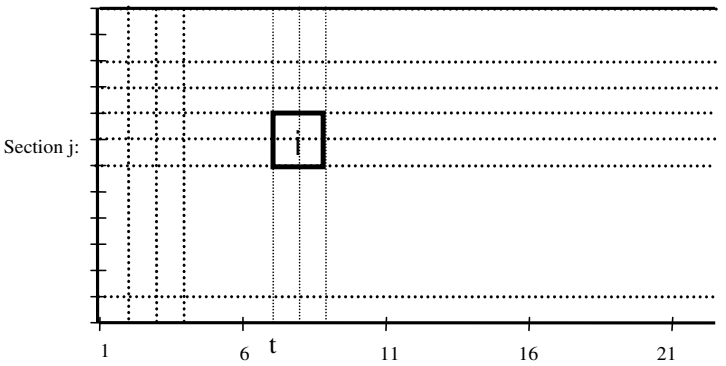


Fig. 3. Position assignment formulation

One feature of RPF is that, as stated in the following proposition, there is an optimal solution in which each vessel rectangle j is immediately right of another vessel rectangle or starting at period a_j, and is immediately above another vessel rectangle or starts at berth section 1.

Proposition 1 *There exists an optimal solution in which for a vessel j, (a) either $u_j = a_j$ or $u_j = u_i + p_i$ for some i; and (b) either $v_j = 1$ or $v_j = v_i + s_i$ for some i.*

Proof. The proof is similar to the proofs of Properties 1–3 in [9] when the penalty costs of not berthing at the optimal locations in their model are not considered and the delay costs for the vessels are set to our weights. □

1.2 Position assignment formulation

An alternative formulation is to consider the space *covered* by the vessel rectangles, where we refer to as the Position Assignment Formulation (PAF). Suppose the space-time area is partitioned into $S \cdot T$ unit blocks (each block has the height of one berth section and the width of one time unit). Let block (m, n) be the block for berth section m, $m \in [1, S]$ and time unit n, $n \in [1, T]$. For instance, in Figure 3, the vessel having a size of 2 and a processing time of 2 units is moored at section j at time t, the blocks (j, t), $(j, t+1)$, $(j+1, t)$, and $(j+1, t+1)$ are covered by this vessel. Let

$$y_{ijt} = \begin{cases} 1, & \text{if the left-bottom corner of vessel rectangle } i \text{ is located} \\ & \text{at section } j \text{ at unit } t, \\ 0, & \text{otherwise;} \end{cases}$$

$$x_{imn} = \begin{cases} 1, & \text{if block } (m, n) \text{ is covered by vessel rectangle } i, \\ 0, & \text{otherwise.} \end{cases}$$

The mathematical formulation can be given as

$$\min \quad \sum_{i=1}^{N} w_i(c_i - a_i) \tag{10}$$

subject to

$$\sum_{j=1}^{S-s_i+1} \sum_{t=a_i}^{T-p_i+1} y_{ijt} = 1 \qquad \forall i \tag{11}$$

$$\sum_{m=j}^{j+s_i-1} \sum_{n=t}^{t+p_i-1} x_{imn} - p_i s_i - (y_{ijt}-1)M \geq 0 \qquad \forall i, j, \text{ and } t \geq a_i \tag{12}$$

$$\sum_{i=1}^{N} x_{imn} \leq 1 \qquad \forall m, n \tag{13}$$

$$p_i + \sum_{j=1}^{S-s_i+1} \sum_{t=a_i}^{T-p_i+1} t y_{ijt} = c_i \qquad \forall i \tag{14}$$

$$x_{imn} \in \{0, 1\} \qquad \forall i, m, n \tag{15}$$

$$y_{ijt} \in \{0, 1\} \qquad \forall i, j, t \tag{16}$$

In this formulation, the objective function is the same as the one of RPF. Constraint (11) shows that each vessel must be moored exactly once. Notice that vessel i must start within sections 1 to $S - s_i + 1$ at a time period between a_i and $T - p_i + 1$. Constraint (12) ensures that the blocks occupied by a particular vessel must be consecutive. Constraint (13) says that a block can only be occupied by at most one vessel rectangle. Constraint (14) relates the processing time, the start time, and the completion time of a vessel. To see it, assume that vessel i starts at time t' at section j', that is, $y_{ij't'} = 1$. By (11), we know that $\sum_{j=1}^{S-s_i+1} \sum_{t=a_i}^{T-p_i+1} t y_{ijt} = t'$. When $j \neq j'$ or $t \neq t'$, we set $y_{ijt} = 0$. Thus, the left hand side of (14) gives us the sum of the mooring time and the processing time of vessel i. By definition, this time is the departure time of vessel i.

1.2.1 A relaxed problem

Suppose that constraint (13) is relaxed and let μ_{mn} be the corresponding Lagrangian multipliers for a given pair of m and n. Let μ be the vector of μ_{mn}. The relaxed problem can be written as

$$L(\mu) \;=\; \min \sum_{i=1}^{N} w_i(c_i - a_i) + \sum_{m=1}^{S} \sum_{n=1}^{T} \mu_{mn} \left(\sum_{i=1}^{N} x_{imn} - 1 \right)$$

subject to (11), (12), (14), (15), and (16).

We now show that the value $L(\mu)$ for a given μ is easy to obtain. Substituting (14) into the objective function and rearranging the terms, the relaxed problem

becomes

$$L(\mu) = \min \sum_{i=1}^{N} \sum_{j=1}^{S-s_i+1} \sum_{t=a_i}^{T-p_i+1} tw_i y_{ijt} + \sum_{m=1}^{S} \sum_{n=1}^{T} \mu_{mn} \sum_{i=1}^{N} x_{imn} + \sum_{i=1}^{N} w_i p_i - \sum_{i=1}^{N} w_i a_i - \sum_{m=1}^{S} \sum_{n=1}^{T} \mu_{mn}$$

subject to (11), (12), (15), and (16).

The last three terms are constants and can be ignored from the optimization point of view. Thus, solving the relaxed problem is equivalent to solving

$$\min \sum_{i=1}^{N} \sum_{j=1}^{S-s_i+1} \sum_{t=a_i}^{T-p_i+1} tw_i y_{ijt} + \sum_{m=1}^{S} \sum_{n=1}^{T} \mu_{mn} \sum_{i=1}^{N} x_{imn} \tag{17}$$

subject to (11), (12) (15), and (16).

Let us consider the second term in the objective function of (17). Since x_{imn} can only be 0 or 1, the sum $\sum_{m=j}^{j+s_i-1} \sum_{n=t}^{t+p_i-1} x_{imn}$ is at most $p_i s_i$. On the other hand, if $y_{ijt} = 1$, then (12) implies

$$\sum_{m=j}^{j+s_i-1} \sum_{n=t}^{t+p_i-1} x_{imn} \geq p_i s_i \quad \forall i, j, t \geq a_i.$$

Therefore, when $y_{ijt} = 1$, we have $x_{imn} = 1$ for $m \in [j, j + s_i + 1]$ and $n \in [t, t + p_i - 1]$. Because our problem is a minimization problem and the multipliers $\mu_{mn} \geq 0$, we know that x_{imn} must be 0 for $m \notin [j, j+s_i+1]$ or $n \notin [t, t+p_i-1]$ in the optimal solution. Thus, for vessel i, if $y_{ijt} = 1$, we have

$$\sum_{m=1}^{S} \sum_{n=1}^{T} \mu_{mn} x_{imn} = \sum_{m=j}^{j+s_i-1} \sum_{n=t}^{t+p_i-1} \mu_{mn}.$$

According to constraint (11), for each i, there is only one $y_{ijt} = 1$ for $j \in [1, S - s_i + 1]$ and $t \in [a_i, T - p_i + 1]$. This implies that the relaxed problem (17) becomes

$$\min \sum_{i=1}^{N} \sum_{j=1}^{S-s_i+1} \sum_{t=a_i}^{T-p_i+1} y_{ijt} \left(tw_i + \sum_{m=j}^{j+s_i-1} \sum_{n=t}^{t+p_i-1} \mu_{mn} \right)$$

subject to (11) and (16) (18)

which is separable in i. Therefore, the solution of the relaxed problem can be obtained, for each i, by setting $y_{ij't'} = 1$ where the pair (j', t') is obtained as

$$(j', t') = \arg \min_{(j, t \geq a_i)} \left(tw_i + \sum_{m=j}^{j+s_i-1} \sum_{n=t}^{t+p_i-1} \mu_{mn} \right). \tag{19}$$

1.2.2 Lower bounds

For fixed values of μ_{mn}, the value $L(\mu)$ gives a lower bound to which we refer to as the Lagrangian Relaxation (LR) bound. To get a better lower bound, we can use the classical subgradient method to determine μ_{mn} such as the one in [5]. In addition to the LR bound, we can obtain lower bounds from the LP relaxations of the two formulations. However, it is well known in the literature that the LR bounds are tighter than the LP bounds.

2 A tree search procedure

In this section, we describe a tree search procedure that utilizes the properties of both RPF and PAF. In the procedure, we add vessel rectangles to the time-space diagram one at a time. If k vessel rectangles have been inserted to the time-space diagram, we call it a k-partial solution. The tree in our procedure has N levels. A node in the tree at level k corresponds to a k-partial solution. The child node of this node is a $(k+1)$-partial solution which is the k-partial solution of its parent node combined with an additional vessel rectangle. Notice that each additional vessel rectangle can be assigned at different positions in the unoccupied space of the diagram. Therefore, a node at level k can have more than $N-k$ child nodes and the corresponding tree has a huge number of nodes. In the following parts, we will describe several approaches to reduce the number of tree nodes, which include efficient branching scheme, tight lower bound for each tree node and duplication reduction.

According to Proposition 1, we can branch a search tree in the following way. The k inserted vessel rectangles form a stair on each tree node. The steps of the stair help indicate potential feasible positions for inserting other vessel rectangles. When we insert a vessel rectangle on one step, the assignment time of the vessel rectangle is the maximum of the step start time and the vessel arrival time. Consider the small example shown in Figure 4a. Suppose that we have inserted vessel rectangle 4 (with $a_4 = 1$, $s_4 = 4$ and $p_4 = 5$) to the left-most bottom position of the time-space diagram, vessel rectangle 1 (with $a_1 = 10$, $s_1 = 2$ and $p_1 = 3$) on the right of vessel rectangle 4 at time 10 (since $a_1 = 10 > 6 = c_4$), and vessel rectangle 3 on the top of vessel rectangle 4. The three insertions result in the 3-partial solution which occupies the left bottom region of the time-space diagram. The boundary of this region forms a stair, as the bold line shown in Figure 4a. This stair has three steps: step 1 starts at time 13 and berth section 1; step 2 starts at time 12 and section 3; and step 3 starts at time 0 and section 10. Suppose that we next insert vessel rectangle 2 (with $a_2 = 11$, $s_2 = 2$ and $p_2 = 3$). According to Proposition 1, vessel rectangle 2 can be put on either step 1, step 2 or step 3. Assume that it is put on step 2. It will start at time $\max\{a_2, \bar{u}_2\} = \bar{u}_2$, where $\bar{u}_2$ represents the start time of the second step. After this inserting operation, another stair with three steps is formed as shown by the dashed bold line in Figure 4a, where the next vessel rectangles can be put on the steps.

To reduce the branching effort, lower and upper bounds are used to provide cuts in the tree searching process. For a given k-partial solution, the LR bound for this

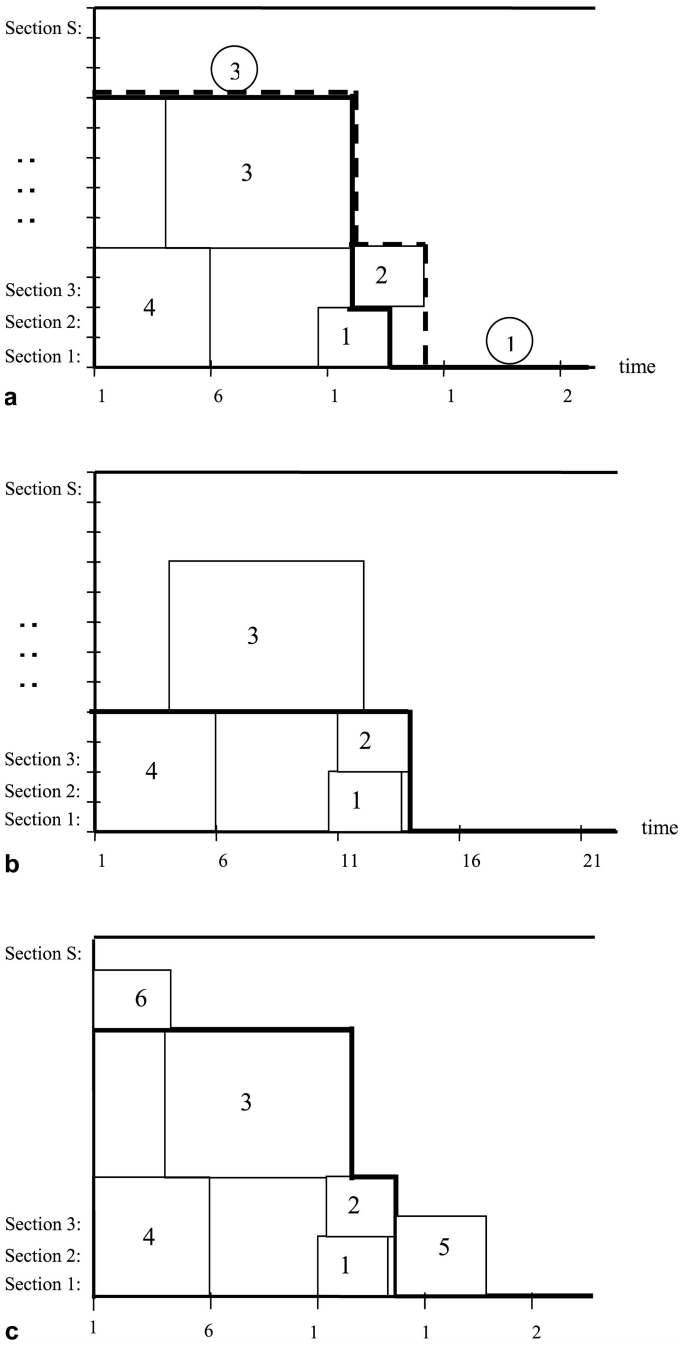


Fig. 4a–c. Tree search procedure

k-partial solution is calculated in such a way that we apply Lagrangian relaxation method to the remaining $N - k$ vessel rectangles on the unoccupied time-space diagram. To get an upper bound, we simply insert the $N - k$ un-inserted vessel rectangles according to the left-most bottom positions each time to obtain a feasible solution. The objective value of this solution represents an upper bound. This upper bound is updated whenever a better feasible solution is observed. When the lower bound of this tree node is larger than the current upper bound, the sub-tree from this node will be pruned.

In our approach, we consider duplication reduction operations on each tree node. For instance, as shown in Figure 4c, consider the six vessel pattern which can be obtained either by the insertion sequence 4-1-2-3-5-6 or the sequence 4-1-2-3-6-5. The two sequences produce duplicated patterns and one will be removed in our tree search approach. When inserting a vessel rectangle on a step of the current stair, we check if the same step exists in the parent node's solution. If so, we insert this vessel rectangle at the same berth section and time unit on the respective steps of the stairs in the partial solution of both the current node and the parent node. We prune the branch from the current node if starting berth section of this vessel rectangle has a smaller value than that of the last inserted vessel rectangle at the current node. For example, if the current node represents the partial solution of 4-1-2-3-6 and its parent node is for 4-1-2-3, and vessel rectangle 5 is to be inserted. This rectangle can be inserted to the position of berth section 1 and time 14. This position is on the current step which exists in both the stairs of the current partial solution 4-1-2-3-6 and that of its parent node 4-1-2-3. Since the starting section number of vessel rectangle 5 is smaller than the starting section number of vessel rectangle 6, the branch 4-1-2-3-6-5 is deleted. Depth-first search strategy is used in our tree search procedure, which helps reduce the memory requirement as a child node differs from its parent node by one vessel rectangle. Therefore, in general, this left-most bottom strategy together with depth-first search approach considerably reduce the need of the complete enumeration.

3 Heuristics

We now propose a heuristic that first groups the vessels into batches according to their arrival times (see assumption 1). For each batch, we use the heuristic of [4] to generate an initial solution. Next, we exchange the vessel rectangles in two consecutive batches if the exchange can reduce the value of the objective function. Then, the exact tree search procedure is applied for each of the resulting batches. Notice that in practice, the number of vessels in each batch is small enough that the tree search procedure can be applied. We review the key steps of the heuristic in [4] in Section 3.1, describe the pair-wise exchange procedure in Section 3.2 and list the steps of the overall procedure in Section 3.3.

3.1 A heuristic for common vessel arrival

When vessels have a common arrival time, the heuristic H of [4] can be applied. In the heuristic, the vessel rectangles are assumed in ascending order of vessel size.

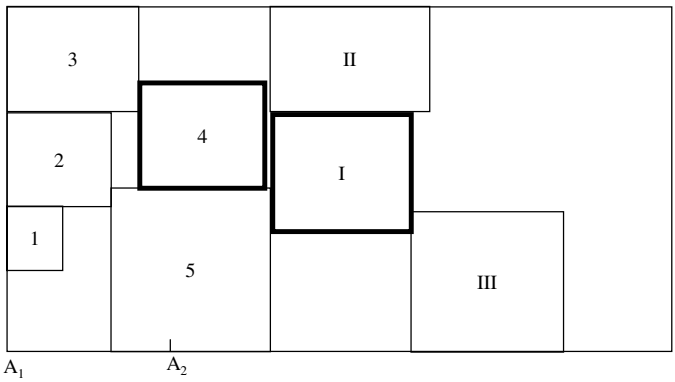
We start by selecting the first group of (smallest) vessel rectangles whose total size is just smaller than S, and assign them to the berth sections. Next, we select the next group and assign them to the berth sections and so on. For an odd group, the vessel rectangles are assigned to the sections according to the increasing order of berth section numbers such that section S will be occupied by the largest vessel rectangle in this group. For an even group, the vessel rectangles are assigned to the sections according to the decreasing order of the section numbers such that section 1 will be occupied by the largest vessel rectangle in this group. Associated with this method is a lower bound (R1) with no more than 100% error determined as follows. First, a vessel rectangle with processing time p_i and size s_i is divided into s_i *sub-rectangles*, each has the height of 1 berth section and the processing time p_i. Then, apply the shortest-processing-time-first (SPT) rule to the sub-rectangles of the relaxed problem. The rule is optimal for the relaxed problem and thus produces a lower bound for the original problem.

3.2 A pair-wise exchange heuristic

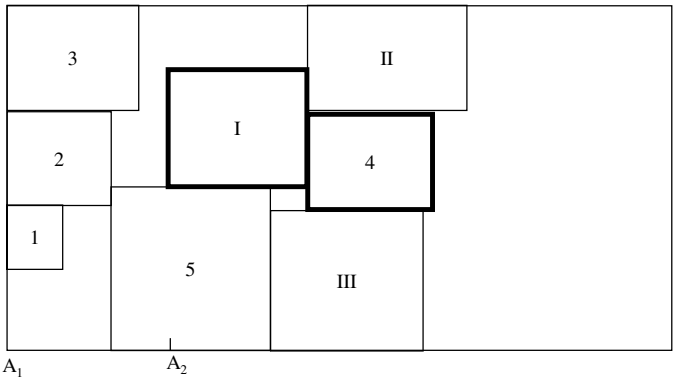
Assume that there are K batches. Let B_k be the set of vessels with arrival times $A_k, k \in [1, K]$ such that $A_1 < A_2 < A_3 < \ldots < A_K$. For two consecutive batches, say B_k and B_{k+1}, the pair-wise exchange heuristic considers a pair of vessel rectangles, one from B_k and one from B_{k+1}. If exchanging the positions of the two vessel rectangles can reduce the objective value, such an exchange will be executed. During the exchange process, other *vessel rectangles'* positions can be moved left, right, up or down to make room for the exchange. After an exchange, we can move other vessel rectangles to the left such that each vessel can be started as early as possible. We illustrate the heuristic using Figure 5. Vessel rectangles $1, 2, 3, 4, 5$ are in B_1 with batch arrival time A_1 and vessel rectangles I, II, III are in B_2 with batch arrival time A_2. Consider the pair vessel rectangle 4 in B_1 and vessel rectangle I in B_2. Initial solution provided by Heuristic H is shown in Figure 5a. Vessel rectangle 4 is right after vessel rectangle 3 and on the top of vessel rectangle 5. Vessel rectangle I is below vessel rectangle II and after vessel rectangle 5. During the exchange operation, we move vessel rectangle II to the right to make it possible to let vessel rectangle I assigned at time A_2, on top of vessel rectangle 5 and before vessel rectangle II. Then, vessel rectangle 4 is put below vessel rectangle II and after vessel rectangle I. Finally, we move vessel rectangle III to the left to reduce the objective value, as shown in Figure 5b.

3.3 A heuristic for batch arrivals

After obtaining an initial solution by Heuristic H for each batch and then applying the pair-wise exchange heuristic, we can use the tree search procedure of Section 2 for each updated batch to further improve the objective value. The process can be repeated until no improvement is found. We call this heuristic for batch arrival as Heuristic HB which has the following steps:



a Schedule before PEH



b Schedule after PEH

Fig. 5a,b. An example for the pair-wise exchange heuristics

Heuristic HB

Step 0: Obtain initial assignment of vessels for each batch B_i, $\forall i \in [1, K]$ by Heuristic H.

Step 1: Sequence B_i, $\forall i \in [1, K]$ according to batch arrival time A_i. Set $i = 1$.

Step 2: For $i < K$, do:

- Step 2.1: For each pair of vessel rectangles with one in B_i and one in B_{i+1}, if the pair-wise exchange heuristic can reduce the objective value, then declare this pair as a candidate. If there is no candidate, go to Step 2.3.
- Step 2.2: Select the candidate pair with the maximum reduction and implement the exchange. Go to Step 2.1.
- Step 2.3: Apply the tree search procedure in B_i and move vessel rectangles in $B_\ell, \ell \in [i+1, K]$ to the left as much as possible. Set $i \leftarrow i+1$ and go to Step 2.

Step 3: If the objective value can be reduced in Step 2, then set i = 1 and go to Step 2. Otherwise, terminate.

4 Numerical experiments

In this section, we investigate the performance of the tree search procedure and Heuristic HB. Section 4.1 describes how the test problems are generated. Section 4.2 reports the tree search procedure performance for small test problems. Section 4.3 shows the performance of the heuristic HB for large test problems. All solution approaches are implemented in C++ using the CPLEX 6.0 library for solving the related linear programs. Experiments were carried out on a Pentium II 300MHz processor PC.

4.1 Random test problems

In our experiment, we set the number of sections S to 10 and the time horizon T to 168, where each unit represents one hour and thus the time horizon is one week. For the common arrival time case, we use different values of N and s_{max} (the maximum vessel length) as described later. Two sets of N numbers are generated independent and identically uniformly distributed in the intervals [6, 12] and [1, s_{max}] to represent the processing times and sizes respectively. These two sets of N numbers are sorted such that $p_1 \leq p_2 \leq \ldots \leq p_N$ and $s_1 \leq s_2 \leq \ldots \leq s_N$. Each pair p_i and s_i is assigned as the processing time and size of vessel i for all i in [1, N]. Hence, vessel sizes and vessel processing times are agreeable.

For the batch arrival time case, we first fix s_{max} to 10 and generate the processing time and size of each vessel as in the above common arrival time case. We use a parameter R_a to indicate different berth utilizations which is defined as $\sum_i p_i s_i/(S \cdot (a_{max} + 12))$, where a_{max} is the latest vessel arrival time and 12 is the longest processing time. Note here a_{max} can be determined by using a given R_a and the generated p_i and s_i. Next, we generate vessel arrival times a_i which are independent and identically uniformly distributed in the interval [0, a_{max}]. Then, we set the number of batches K to be the rounded-up integer of $a_{max}/12$ and set batch arrival times A_k to be 0, 12, 24, ... etc. Finally, we group vessel i to batch k if $a_i \geq A_k$ and $a_i < A_{k+1}$ and update vessel arrival time a_i to be A_k.

4.2 Exact solution methods

There are three exact solution methods for the berth allocation problem. They are CPLEX solutions for RPF and PAF and the developed tree search procedure as described in Section 2. When implementing the tree search procedure, a larger value of $k_{\max}$ (the number of iterations in the subgradient method) will produce a tighter LR bound and thus the tree search requires a smaller number of nodes. However, it takes a longer CPU time. We did some preliminary experiments and found that the overall CPU time was smaller when the value of $k_{\max}$ was between 5 and 10. Thus, we set $k_{\max} = 10$ for problems with less than 10 vessels and $k_{\max} = 5$ otherwise. In this experiment, we fix $s_{max} = S$. As the formulations are integer programs, CPLEX employs the branch-and-bound method that involves a large number of tree nodes. Direct application of CPLEX to PAF cannot provide

Table 1. Computational results for the tree search procedure and CPLEX

N	RPF (by CPLEX)		Tree search	
	#nodes	CPU(sec)	#nodes	CPU(sec)
4	83	0.11	34	0.11
10	244.4K	660	1.9K	49
15	–	–	120.4K	55325

an optimal solution for the cases that have more than 4 vessels. In the experiment, N is set to be 4, 10, 15 respectively. We take five samples for each value of N and record the average value, as shown in Table 1. The tree search procedure requires a substantially smaller number of tree nodes than the branch-and-bound method. Consequentially, the tree search procedure can solve problems up to 15 vessels. In practice, the largest berth in Hong Kong's terminals can hold up to four ocean going vessels where the processing time for a vessel typically ranges from 6 to 12 hours. Thus, if we have one batch per day, then the tree search procedure can still be used.

4.3 Performance of the heuristics

First, we would like to evaluate the performance of Heuristic H for the common arrival time case as there is no numerical results in [4]. We set N to be 5, 10, 15, ..., 80, and s_{max} to be 2, 3, ..., 7. For each combination, 100 random samples are generated. For each sample, we obtain the objective value Z^H by Heuristic H, lower bound R1 (see Section 3.1) and the Lagrangian relaxation lower bound LR. Average and maximum ratios of Z^H to each lower bound for each combination are shown in Table 2. Computational results show the robust performance of Heuristic H. The worst case sample error is around 25% when $N = 10$ and $s_{max}/S = 0.7$, and the average error is no more than 15%. It is observed that Heuristic H performs better when $s_{\max}$ is small. Computational results also show that LR provides tighter lower bounds than R1 as the ratio $s_{\max}/S$ increases and the latter provides tighter lower bound only when the ratio is very small. In our experiment, Heuristic H took only 0.06 second for the largest problem.

Second, we would like to evaluate the effectiveness of Heuristic HB. We create different combinations by setting $R_a = 0.6, 0.75, 0.9$ and N=12, 18, ... , 42 respectively. For each combination, we apply Heuristic H for each batch, then use the pair-wise exchange heuristic to swap vessel rectangles between batches, and finally apply the tree search procedure. Let Z^H, Z^P, and Z^T be the objective values resulting from these three procedures respectively. Let Z_l be the LR bound. We take 30 samples for each combination and report the average and maximum gaps for the three procedures, defined as (Z^H-Z_l)/Z^H, (Z^P-Z_l)/Z^P and (Z^T-Z_l)/Z^T which are indicated by H, PE, and TS in Table 3 respectively. Experiment results indicate that the pair-wise exchange heuristic and the tree search procedure can reduce the error

Table 2. Computational results for the common arrival time case

S_{max}/S	Z^H/Z_u	N=10		N=20		N=40		N=80	
		Ave	Max	Ave	Max	Ave	Max	Ave	Max
0.2	R1	1.02	1.03	1.03	1.07	1.02	1.03	1.01	1.02
	LR	1.03	1.05	1.05	1.07	1.02	1.04	1.02	1.03
0.3	R1	1.10	1.14	1.06	1.09	1.06	1.08	1.05	1.07
	LR	1.10	1.13	1.05	1.06	1.04	1.05	1.04	1.05
0.4	R1	1.12	1.19	1.14	1.21	1.12	1.17	1.11	1.17
	LR	1.09	1.14	1.09	1.12	1.08	1.10	1.07	1.10
0.5	R1	1.12	1.22	1.11	1.17	1.12	1.14	1.11	1.17
	LR	1.10	1.15	1.08	1.11	1.08	1.09	1.08	1.11
0.6	R1	1.23	1.36	1.19	1.23	1.20	1.26	1.18	1.31
	LR	1.15	1.24	1.10	1.12	1.10	1.14	1.11	1.13
0.7	R1	1.26	1.39	1.24	1.41	1.28	1.32	1.27	1.38
	LR	1.15	1.24	1.10	1.15	1.13	1.15	1.14	1.20

Table 3. Error gaps produced by the steps of heuristics HB for the batch arrival case

		Ra=0.6		Ra=0.75		Ra=0.9	
#vessels		Ave	Max	Ave	Max	Ave	Max
12	H	0.12	0.18	0.28	0.4	0.29	0.4
	PE	0.11	0.15	0.21	0.31	0.15	0.29
	TS	0.05	0.08	0.09	0.19	0.06	0.11
18	H	0.33	0.44	0.37	0.45	0.4	0.5
	PE	0.23	0.32	0.26	0.32	0.24	0.31
	TS	0.05	0.08	0.08	0.15	0.07	0.1
24	H	0.44	0.56	0.45	0.58	0.49	0.56
	PE	0.26	0.32	0.24	0.36	0.27	0.33
	TS	0.11	0.14	0.09	0.14	0.12	0.19
30	H	0.46	0.54	0.5	0.58	0.53	0.62
	PE	0.28	0.38	0.29	0.41	0.24	0.32
	TS	0.14	0.19	0.11	0.13	0.15	0.19
36	H	0.54	0.63	0.55	0.65	0.57	0.65
	PE	0.25	0.32	0.34	0.41	0.23	0.38
	TS	0.16	0.22	0.17	0.22	0.12	0.18
42	H	0.54	0.59	0.59	0.62	0.6	0.66
	PE	0.23	0.32	0.26	0.35	0.29	0.34
	TS	0.17	0.23	0.17	0.2	0.16	0.21

gaps significantly. Average relative errors for batch arrival time case are around 10%–15%. The CPU times for Heuristic H, the pair-wise exchange heuristic and the tree search procedures are 0.05 second, 26.25 seconds and 1671.81 seconds respectively for the largest problem where $R_a = 0.9$ and $N = 42$. This indicates our solution approaches are very efficient in solving the berth allocation problem.

5 Summary

In this paper, we study a berth allocation model that allows multiple vessel mooring per berth, considers vessel arrival times, and has the objective of minimizing the total weighted flow time. In our solution approach, we assume vessel arrivals can be grouped into batches. After describing the *berth allocation problem* and reviewing current researches on this problem, we consider two formulations with which several bounds are derived. Next, a tree search procedure is described which provides an exact solution and it performs better than the direct application of CPLEX. Then, we develop a composite heuristic HB that combines Heuristic H of [4], a pair-wise exchange heuristic and the tree search procedure. The computational experiments show that Heuristic H is efficient to generate good initial solutions while the composite heuristic HB is quite effective.

In the future research, we may consider different objectives, i.e., vessel tardiness and weighted vessel tardiness when vessel due dates are given. In this research, we make the assumption that vessel processing times are constant in the models. In practice, vessel processing time can be adjusted by changing the number of cranes that are operating on the vessel. In this situation, the number of cranes on berth assigned for each vessel can be considered as a decision variable.

References

1. Chen C-Y, Hsieh T-W (1998) A time-space network model for the berth allocation problem. First International Conference of Maritime Engineering and Ports, Genoa
2. Brown G, Cormican K, Lawphogpanich S, Widdis D (1997) Optimizing submarine berthing with a persistence incentive. Naval Research Logistics **44**, 301–318
3. Daganzo C (1989) The crane scheduling problem. Transportation Research B **23B**, 159–175
4. Guan Y, Xiao W-Q, Cheung RK, Li C-L (2002) A multiprocessor task scheduling model for berth allocation: heuristic and worst case analysis. Operations Research Letters **30**, 343–350
5. Held M, Karp RM (1971) The traveling salesman problem and minimum spanning trees: Part II. Mathematical Programming **1**, 6–25
6. Hadjiconstantinous E, Christofides N (1995) An exact algorithm for general, orthogonal, two-dimensional knapsack problems. European Journal of Operational Research **83**, 39–56
7. Imai A, Nagaiwa K, Chan WT (1997) Efficient planning of berth allocation for container terminals in Asia. Journal of Advanced Transportation **31**, 75–94
8. Imai A, Nishimura E, Papadimitriou S (2001) The dynamic berth allocation problem for a container port. Transportation Research B **35**, 401–417

9. Kim KH, Moon KC (2003) Berth scheduling by simulated annealing. Transportation Research B **37(6)**, 541–560
10. Lai KK, K Shih (1992) A study of container berth allocation. Journal of Advanced Transportation **26**, 45–60
11. Li C-L, Cai X, Lee C-Y (1998) Scheduling with multiple-job-on-one-processor pattern. IIE Transactions **30,** 433–445
12. Lim A (1998) The berth planning problem. Operations Research Letters **22**, 105–110
13. Peterkofsky RI, Daganzo CF (1990) A branch and bound solution method for the crane scheduling problem. Transportation Research B **24B**(3), 159–172
14. Sim CW, Lin LS, Soon NH (1992) Berth allocation in the port of Singapore Authority: a total approach. Proceedings of the First Singapore International Conference on Intelligent Systems, 237–241